Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática – ICEI
Arquitetura de Computadores I

ARQ1 _ Aula_13
Tema: Introdução à linguagem Verilog e simulação em Logisim (circuitos sequenciais)

Orientação geral:

Apresentar todas as soluções em apenas um arquivo com formato texto (.txt).
Sugere-se usar como nome Guia_xx.txt, onde xx indicará o guia, exemplo Guia_01.txt.
Todos os arquivos deverão conter identificações iniciais com o nome e matrícula,
no caso de programas, usar comentários.
As implementações e testes dos exemplos em Verilog (**.v**) fornecidos como pontos de partida,
também fazem parte da atividade e deverão ter os códigos fontes entregues **separadamente**,
com o código fonte e os módulos de testes, a fim de que possam ser compilados e verificados.
Sugere-se usar como nomes Guia_01yy.v, onde yy indicará a questão, exemplo Guia_0101.v
As saídas de resultados, opcionalmente, poderão ser copiadas ao final do código, em comentários.
Quaisquer outras anotações, observações ou comentários poderão ser colocadas em arquivo texto (README.txt) acompanhando a entrega.

Outras formas de solução serão **opcionais**; não servirão para substituir as atividades
a serem avaliadas. Caso entregues, poderão contar apenas como atividades extras.
Os programas com funções desenvolvidas em C, Java ou Python (c, .java, py),
como os modelos usados para verificação automática de testes das respostas;
caso entregues, também deverão estar em arquivos **separados**, com o código fonte e
e os módulos de testes, a fim de que possam ser compilados e verificados.
As execuções deverão, preferencialmente, serão testadas mediante uso de redirecionamento de entradas e saídas padrões, cujos dados/resultados deverão ser armazenados em arquivos textos.
Os resultados poderão ser anexados ao código, ao final, como comentários.
Planilhas, caso venham a ser solicitadas, deverão ser **programadas** e/ou usar funções nativas.
Serão suplementares e opcionais, e deverão ser entregues em formato texto, preferencialmente,
com colunas separadas por tabulações ou no formato (**.csv**), acompanhando a solução em texto.
Arquivos em formato (**.pdf**), fotos, cópias de tela ou soluções manuscritas também poderão ser aceitos como recursos suplementares para visualização, mas **não** servirão como substitutos e **não** terão validade para fins de avaliação.
Os *layouts* de circuitos deverão ser entregues no formato (**.circ**), identificados internamente.
Os *layouts* de diagramas deverão ser entregues no formato (**.jff**), identificados externamente.
Figuras exportadas pela ferramenta serão aceitas apenas como arquivos para visualização,
mas **não** terão validade para fins de avaliação. Separar versões completas (a) e simplificadas (b).

Atividade: Circuitos sequenciais – *Flip-Flops* – Contadores

Análise e síntese de circuitos sequenciais

As técnicas para análise de circuitos sequenciais que implementam uma certa máquina de estados finitos, em geral, dividem-se em duas etapas:

1. determinar as funções que determinam o próximo estado e as saídas

   1.1 especificar as equações que representem a lógica do circuito e as saídas de cada ***flip-flop*** (estado corrente);

   1.2 especificar as equações que determinem as transições entre dois pulsos de ***clock***;

   1.3 construir a *tabela de transições* para cada uma das combinações das entradas, indicando quais os próximos estados;

   1.4 identificar todas as combinações que representem um mesmo estado e reescrevê-las em uma *tabela de estados*;

2. construir as tabelas de estados/saídas que especifiquem o comportamento do circuito para todas as combinações das entradas e do estado corrente:

   2.1 verificar as funções das saídas em relação às entradas e aos estados correntes;

   2.2 após avaliar todas as combinações de entradas e estados, combinar a tabela de estados com essas informações e criar a tabela de estados/saídas, relacionando cada saída ao próximo estado.

Exemplo 1:

Considerar o circuito abaixo com um ***flip-flop*** tipo D.

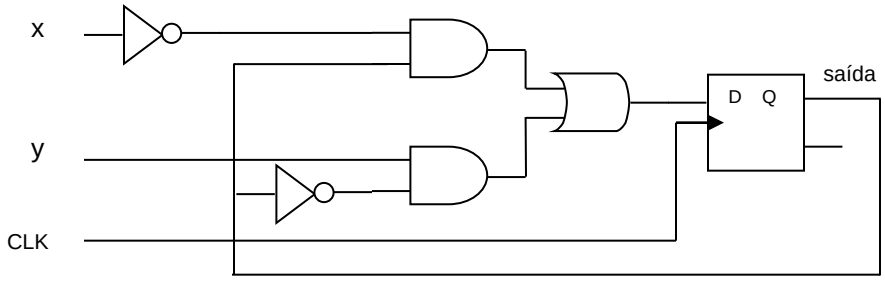


Tabela de transições

Q_t\xy	00	01	10	11
0	0	1	0	1
1	1	1	0	0

Q_{t+1}

Equações de transições

$D = x' \bullet Q + y \bullet Q'$

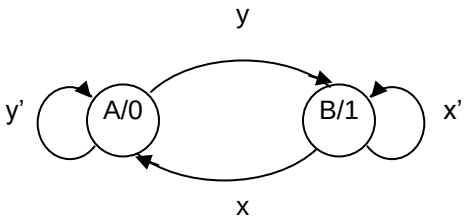
$Q_{t+1} = x' \bullet Q_t + y \bullet Q'_t$

Tabela de estados/saídas

Q_t\xy	00	01	10	11
A	A,0	B,1	A,0	B,1
B	B,1	B,1	A,0	A,0

estados Q_{t+1}, saída

Diagrama de estados



Considerar o circuito abaixo com dois ***flip-flops*** tipo JK.

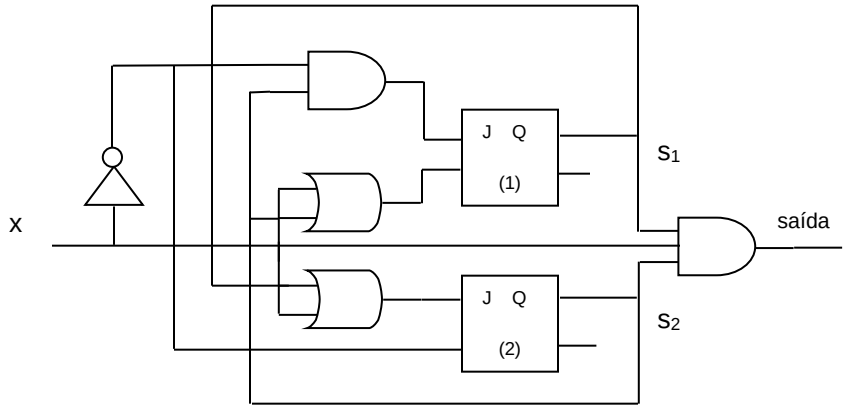


Tabela de transições

s_1 (t)	s_2 (t)	x	s_1 (t+1)	s_2 (t+1)	saída
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	0	1	0
1	1	0	0	0	0
1	1	1	0	1	1

Equações de transições

saída = $s_1 \bullet s_2 \bullet x$

$J_1 = s_2 \bullet x'$ e $K_1 = s_2 + x$

$J_2 = s_1 + x$ e $K_2 = x'$

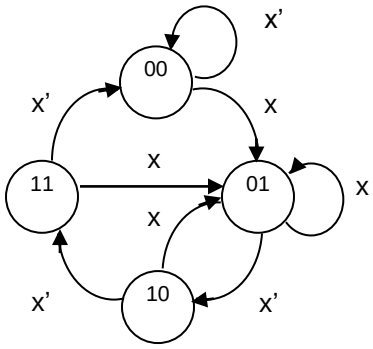
$$Q_{t+1} = J_1 Q'_t + K_1' Q_t$$
$$s_1 = s_2 \bullet x' \bullet s_1' + (s_2 + x)' \bullet s_1$$
$$= s_2 \bullet x' \bullet s_1' + s_2' \bullet x' \bullet s_1$$
$$= x' \bullet (s_2 \bullet s_1' + s_2' \bullet s_1)$$
$$= x' \bullet (s_1 \textbf{ xor } s_2)$$

$$Q_{t+1} = J_2 Q'_t + K_2' Q_t$$
$$s_2 = (x+s_1) \bullet s_2' + (x')' \bullet s_2$$
$$= (x \bullet s_2') + (s_1 \bullet s_2') + (x \bullet s_2)$$
$$= x \bullet (s_2' + s_2) + (s_1 \bullet s_2')$$
$$= x + (s_1 \bullet s_2')$$

Tabela de estados/saídas

s_1	s_2	x=0		x=1		saída
0	0	0	0	0	1	0
0	1	1	0	0	1	0
1	0	1	1	0	1	0
1	1	0	0	0	1	0/1

Diagrama de estados

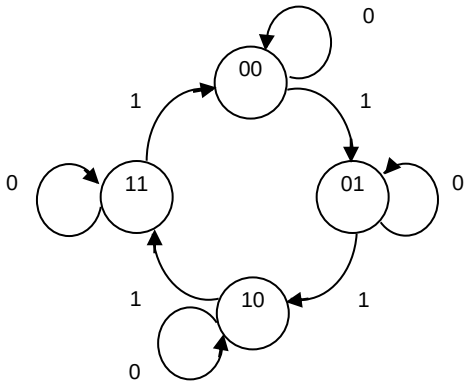


Exemplo 2:

Projetar um contador crescente módulo 4 (0-1-2-3-0) com ***flip-flops*** tipo D.

Tabela de transições

Diagrama de estados

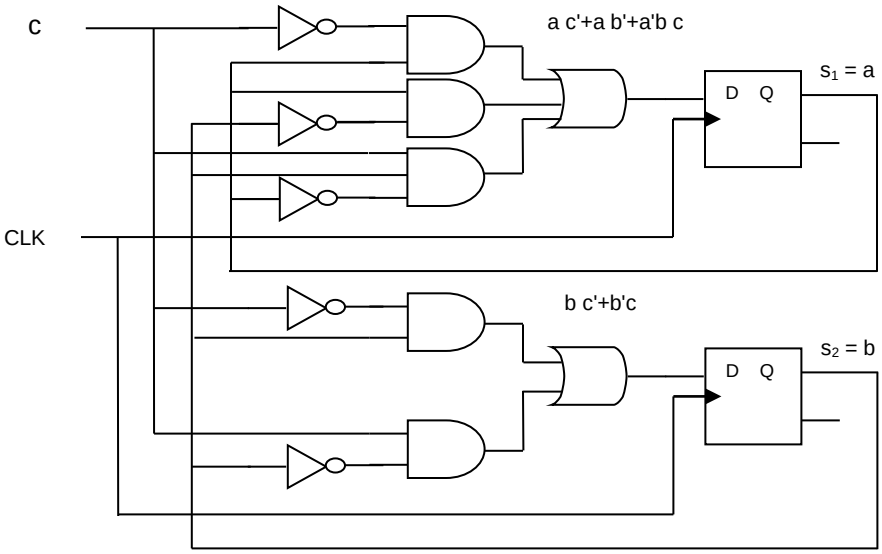


	s_1 (t)	s_2 (t)	evento	s_1 (t+1)	s_2 (t+1)
	a	b	c	a	b
0	0	0	0	0	0
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	1	0
4	1	0	0	1	0
5	1	0	1	1	1
6	1	1	0	1	1
7	1	1	1	0	0

Equações de transições

sinais	SoP	mintermos	simplificação
s_1	3,4,5,6	a'bc+ab'c'+ab'c+abc'	ac'+ab'+ab'c
s_2	1,2,5,6	a'b'c+a'bc'+ab'c+abc'	bc'+b'c

Circuito

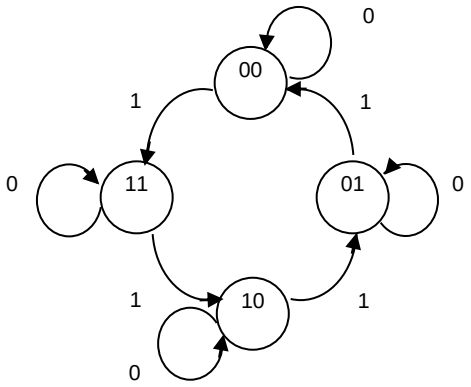


Exemplo 3:

Projetar um contador decrescente módulo 4 (0-3-2-1-0) com ***flip-flops*** tipo D.

Tabela de transições

Diagrama de estados

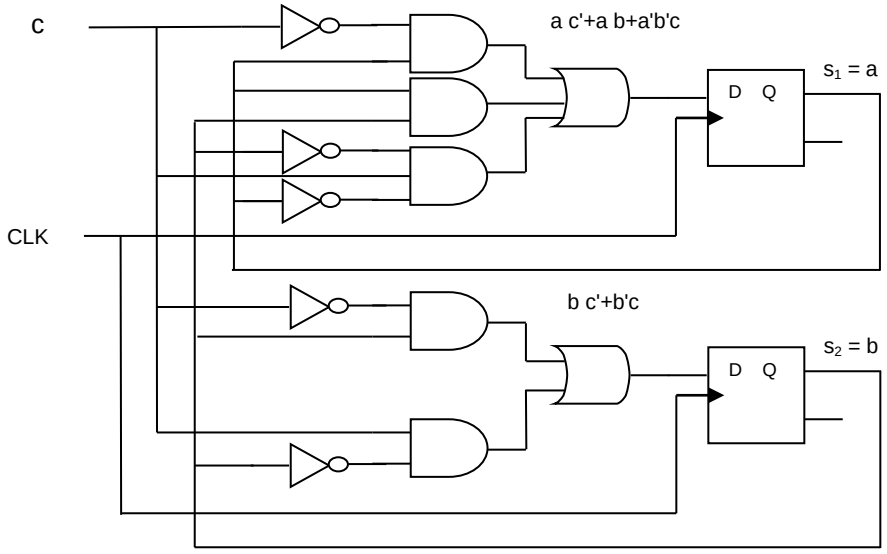


	s_1 (t)	s_2 (t)	evento	s_1 (t+1)	s_2 (t+1)
	a	b	c	a	b
0	0	0	0	0	0
1	0	0	1	1	1
2	0	1	0	0	1
3	0	1	1	0	0
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	1	1
7	1	1	1	1	0

Equações de transições

sinais	SoP	mintermos	simplificação
s_1	1,4,6,7	a'b'c+ab'c'+abc'+abc	ac'+ab+a'b'c
s_2	1,2,5,6	a'b'c+a'bc'+ab'c+abc'	bc'+b'c

Circuito



Exercícios

01.) Projetar e descrever em Logisim e Verilog por portas um módulo,
com portas e flip-flops tipo JK apenas,
para implementar um contador assíncrono decrescente
com 6 bits de comprimento.
Dica: Usar modelo anexo. Não usar arranjo.

02.) Projetar e descrever em Logisim e Verilog por portas um módulo
com portas e flip-flops tipo JK apenas,
para implementar um contador assíncrono crescente
com 6 bits de comprimento.

03.) Projetar e descrever em Logisim e Verilog por portas um módulo,
com portas lógicas e flip-flops tipo JK apenas,
para implementar um contador assíncrono decádico crescente
com 5 bits de comprimento.
Dica: Usar modelo anexo. Não usar arranjo.

04.) Projetar e descrever em Logisim e Verilog por portas um módulo
com portas e flip-flops tipo JK apenas,
para implementar um contador assíncrono decádico decrescente
com 5 bits de comprimento.

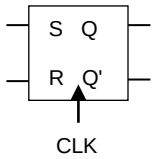
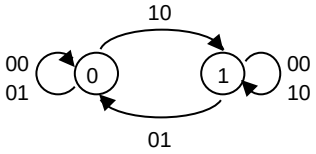
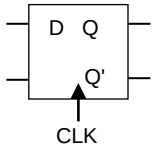
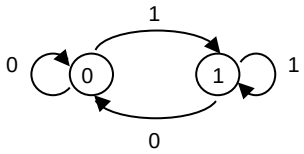
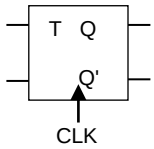
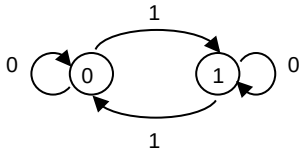
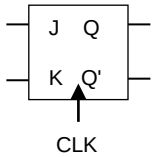
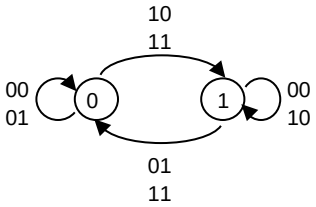
05.) Projetar e descrever em Logisim e Verilog um módulo,
com portas e flip-flops tipo T apenas,
para implementar um contador síncrono módulo 7.
Dica: Usar modelo anexo. Não usar arranjo.

Extras

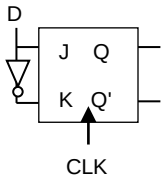
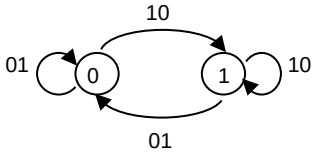
06.) Projetar e descrever em Logisim e Verilog por portas um módulo,
com portas e flip-flops tipo JK apenas,
para implementar um contador em anel com 6 bits de comprimento.
Dica: Usar modelo anexo. Não usar arranjo.

07.) Projetar e descrever em Logisim e Verilog por portas um módulo
com portas e flip-flops tipo JK apenas,
para implementar um contador em anel torcido com 6 bits de comprimento.
Dica: Usar modelo anexo. Não usar arranjo.

Flip-flops

Flip-flop	Estados	Característica	Transição	Equação
		S R Q_{t+1} Q'_{t+1} 0 0 Q_t Q_t' 0 1 0 1 1 0 1 0 1 1 ? ?	Q_t Q_{t+1} S R 0 0 0 X 0 1 1 0 1 0 0 1 1 1 X 0	$Q_{t+1}=S+R'.Q_t$
		D Q_{t+1} Q'_{t+1} 0 0 1 1 1 0	Q_t Q_{t+1} D 0 0 0 0 1 1 1 0 0 1 1 1	$Q_{t+1} = D$
		T Q_{t+1} Q'_{t+1} 0 Q_t Q_t' 1 Q_t' Q_t	Q_t Q_{t+1} T 0 0 0 0 1 1 1 0 1 1 1 0	$Q_{t+1} = T \oplus Q_t$
		J K Q_{t+1} Q'_{t+1} 0 0 Q_t Q_t' 0 1 0 1 1 0 1 0 1 1 Q_t' Q_t	Q_t Q_{t+1} J K 0 0 0 X 0 1 1 X 1 0 X 1 1 1 X 0	$Q_{t+1}=J.Q_t'+K'.Q_t$

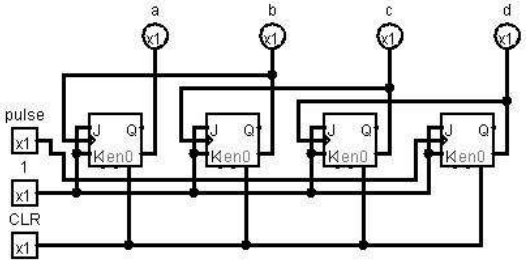
Configurações especiais

Flip-flop	Estados	Característica	Transição	Equação
		J K Q_{t+1} Q'_{t+1} 0 1 0 1 1 0 1 0

```
module dff ( output q, output qnot,
                 input   d, input clk,
                 input   preset, input clear );
reg q, qnot;

always @( posedge clk )
begin
  if ( clear )      begin q <= 0; qnot <=   1; end
  else
    if ( preset )  begin q <= 1; qnot <=   0; end
    else
                   begin  q <= d; qnot <= ~d; end
end

endmodule // dff

module jkff ( output q, output qnot,
            input  j, input k,
            input clk, input preset, input clear );

reg    q, qnot;

always @( posedge clk or
                posedge preset or
                posedge clear )
begin
  if ( clear )      begin q <= 0; qnot <= 1; end
  else
    if ( preset )  begin q <= 1; qnot <= 0; end
    else
      if ( j & ~k ) begin q <= 1; qnot <= 0; end
      else
       if ( ~j & k ) begin q <= 0; qnot <= 1; end
       else
        if ( j & k )
             begin q <= ~q; qnot <= ~qnot; end
end

endmodule // jkff
```

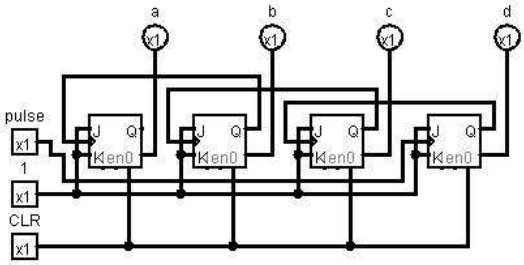
```
module tff ( output q, output qnot,
                 input    t, input    clk,
                 input    preset, input clear );

reg q, qnot;

always @( posedge clk or ~preset or ~clear)
begin
 if ( ~clear )
  begin    q <= 0;            qnot <= 1;   end
 else
  if ( ~preset )
   begin   q <= 1;            qnot <= 0;   end
  else
   begin
    if ( t ) begin  q <= ~q; qnot <= ~qnot; end
   end
end

endmodule // tff

module srff ( output q, output qnot,
                  input    s, input  r, input clk,
                  input preset, input clear );
reg q, qnot;

always @( posedge clk )
begin
  if ( clear )        begin q <= 0; qnot <= 1; end
  else
    if ( preset )    begin q <= 1; qnot <= 0; end
    else
      if ( s & ~r )   begin q <= 1; qnot <= 0; end
      else
        if ( ~s & r ) begin q <= 0; qnot <= 1; end
        else
          if ( s & r )
           begin   q <= 0; qnot <= 0;   end // arbitrary
end

endmodule // srff
```

(Down) Asynchronous counter



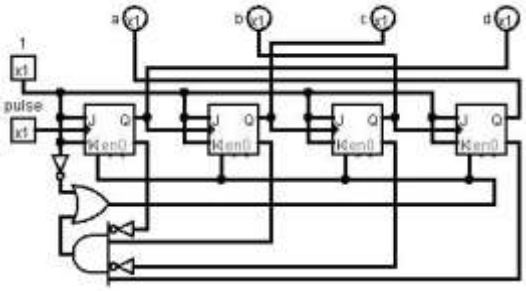
OPS: CLR - 1 - pulse

(Up) Asynchronous counter

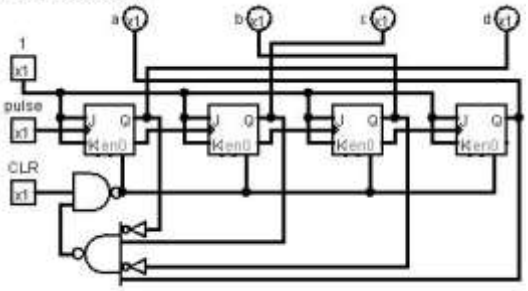


OPS: CLR - 1 - pulse

(Down) Decade counter

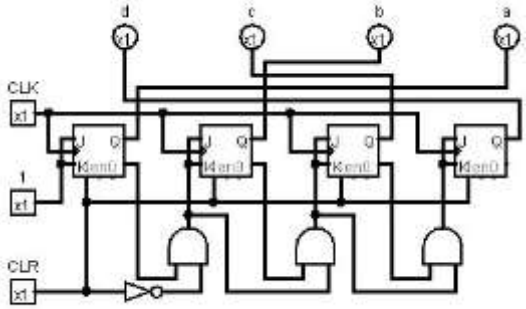


(Up) Decade counter



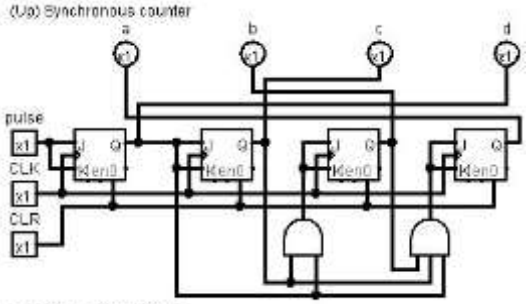
OPS: CLR - 1 - pulse

(Down) Synchronous counter



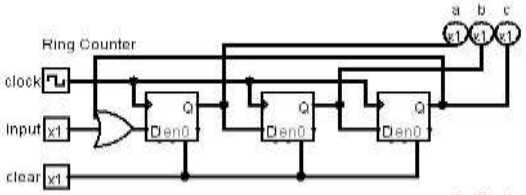
OPS: CLR - 1 - CLK

(Up) Synchronous counter

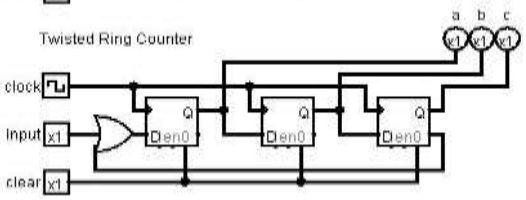
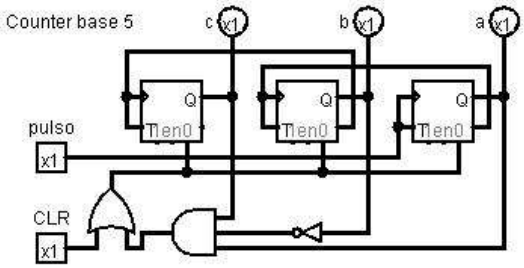


OPS: CLR - pulse - CLK

Ring Counter



Twisted Ring Counter



Counter base 5